

MC60 OpenCPU

GS3 SDK — Fourth Test Report (Cell Tower Location)

SDK MC60_OpenCPU_GS3_SDK_V1.8

Toolchain Arm GNU Toolchain 15.2.1

Host OS Windows 11

Date July 2026

Example example_location.c (QuecLocator)

Interface PuTTY via CP2102 (COM6, 115200 baud)

Internal documentation — not for distribution

Contents

1	Introduction	2
2	How example_location.c Works	2
3	Build	3
4	Flash	4
5	Serial Output and Analysis	4
5.1	Boot and Network Registration	4
5.2	PDP Context and QuecLocator	5
5.3	Location Result	5
6	Location Verification	6
7	Summary	7
	References	8

1. Introduction

This report documents the fourth test of the Quectel MC60 module using the MC60_OpenCPU_GS3_SDK_V1.8. The previous three tests covered the GPS example (no fix, then valid outdoor fix) and the Bluetooth example. This test uses a fundamentally different positioning method: **cell tower triangulation via QuecLocator**, enabled by the example_location.c example and a live SIM card.

Instead of relying on GNSS satellite signals, QuecLocator uses the GSM/GPRS network to which the SIM is registered. The module connects to Quectel's QuecLocator server over a GPRS data bearer and sends the identifiers of visible cell towers. The server triangulates a position from its cell-tower database and returns latitude and longitude coordinates. This approach works indoors and requires no GPS antenna — only a registered SIM and a GPRS data connection.

The toolchain, flash procedure, and serial monitor setup established during the first test were reused without modification. The only change from previous tests was the active preprocessor example flag and the presence of a live SIM card in the EVB.

2. How example_location.c Works

example_location.c implements the OpenCPU location API demo. On startup it runs Location_Program(), which executes the following sequence automatically:

- **RIL initialisation:** Waits for the Radio Interface Layer to be ready before proceeding.
- **GSM registration:** Polls GSM network status until the module registers on the home network (status 2).
- **GPRS registration:** Waits for GPRS data network registration (status 2), required for a data bearer.
- **SIM check:** Confirms the SIM card is present and ready (no PIN required).
- **PDP context setup:** Selects PDP context 0, sets the APN, and activates the GPRS data bearer (RIL_NW_SetAPN + RIL_NW_ActivePDPContext).
- **QuecLocator query:** Calls RIL_GetLocation, which contacts Quectel's QuecLocator server over the active GPRS bearer and returns latitude and longitude via cell-tower triangulation.
- **Result output:** Prints the returned coordinates over the debug UART and terminates.

Key difference from the GPS tests

GPS (tests 1 & 3): GNSS receiver + satellite signals + external antenna
QuecLocator (this test): SIM card + GPRS data bearer + Quectel server
No GPS antenna or satellite acquisition time required.
Accuracy: cell-tower triangulation is typically 50-500 m (vs sub-10 m for GPS).

3. Build

To switch from the GPS example to the location example, the preprocessor define in `gcc_makefile` was updated:

```
C_PREDEF=-D __CUSTOMER_CODE__ -D __EXAMPLE_LOCATION__
```

The project was then rebuilt from a clean state using the SDK build script:

```
.\Make.bat new
```

The same `gcc_makefile` / `gcc_makefiledef` pair fixed during the first test — with the Arm GNU Toolchain 15.2.1 path, updated library paths, and warning-suppression flags — was used without any further modification. All SDK example source files, including `example_location.c` and `example_location2.c`, compiled successfully. Figure 1 shows the tail end of the build output confirming successful compilation.

```

- Building build\gcc\obj/example/example_ble_client.o
- Building build\gcc\obj/example/example_bluetooth.o
- Building build\gcc\obj/example/example_call.o
- Building build\gcc\obj/example/example_clk.o
- Building build\gcc\obj/example/example_csd.o
- Building build\gcc\obj/example/example_download_epo.o
- Building build\gcc\obj/example/example_dtmf.o
- Building build\gcc\obj/example/example_eint.o
- Building build\gcc\obj/example/example_epo_callback.o
- Building build\gcc\obj/example/example_file.o
- Building build\gcc\obj/example/example_float_math.o
- Building build\gcc\obj/example/example_fota_ftp.o
- Building build\gcc\obj/example/example_fota_http.o
- Building build\gcc\obj/example/example_ftp.o
- Building build\gcc\obj/example/example_gpio.o
- Building build\gcc\obj/example/example_gps.o
- Building build\gcc\obj/example/example_gps_iic.o
- Building build\gcc\obj/example/example_http.o
- Building build\gcc\obj/example/example_iic.o
- Building build\gcc\obj/example/example_led.o
- Building build\gcc\obj/example/example_location.o
- Building build\gcc\obj/example/example_location2.o
- Building build\gcc\obj/example/example_memory.o
- Building build\gcc\obj/example/example_mqtt.o
- Building build\gcc\obj/example/example_multitask.o
- Building build\gcc\obj/example/example_multitask_port.o
- Building build\gcc\obj/example/example_pwm.o
- Building build\gcc\obj/example/example_sms.o
- Building build\gcc\obj/example/example_spi.o
- Building build\gcc\obj/example/example_system.o
- Building build\gcc\obj/example/example_tcp_demo.o
- Building build\gcc\obj/example/example_tcpclient.o
- Building build\gcc\obj/example/example_tcpserver.o
- Building build\gcc\obj/example/example_time.o
- Building build\gcc\obj/example/example_timer.o
- Building build\gcc\obj/example/example_transpass.o
- Building build\gcc\obj/example/example_udpclient.o
- Building build\gcc\obj/example/example_udpserver.o
- Building build\gcc\obj/example/example_watchdog.o
- Building build\gcc\obj/example/example_nema_pro.o
- Building build\gcc\obj/example/example_utility.o
- Building build\gcc\obj/custom/fota/src/fota_ftp.o
- Building build\gcc\obj/custom/fota/src/fota_http.o
- Building build\gcc\obj/custom/fota/src/fota_http_code.o
- Building build\gcc\obj/custom/fota/src/fota_main.o

- GCC Compiling Finished Successfully.
- The target image is in the 'build\gcc' directory.

make.exe[1]: Leaving directory 'C:\Users\Ali\Downloads\Compressed\SDK\MC60_OpenCPU_GS3_SDK_V1.8'
PS C:\Users\Ali\Downloads\Compressed\SDK\MC60_OpenCPU_GS3_SDK_V1.8>

```

Figure 1: PowerShell build output — GCC compiling all SDK example objects including example_location.o and example_location2.o, finishing with “GCC Compiling Finished Successfully” and the binary placed in build\gcc\.

4. Flash

The updated binary was flashed to the MC60 using **QFlash V4.0**, with the same configuration as all previous tests: COM 6, baud rate 460800 for the download, module type MC60, and the three required files (SCAT scatter, DA download agent, APPGS3MDM32A01.bin application binary). The flash operation completed with a **PASS** status in 32 seconds, as shown in Figure 2.

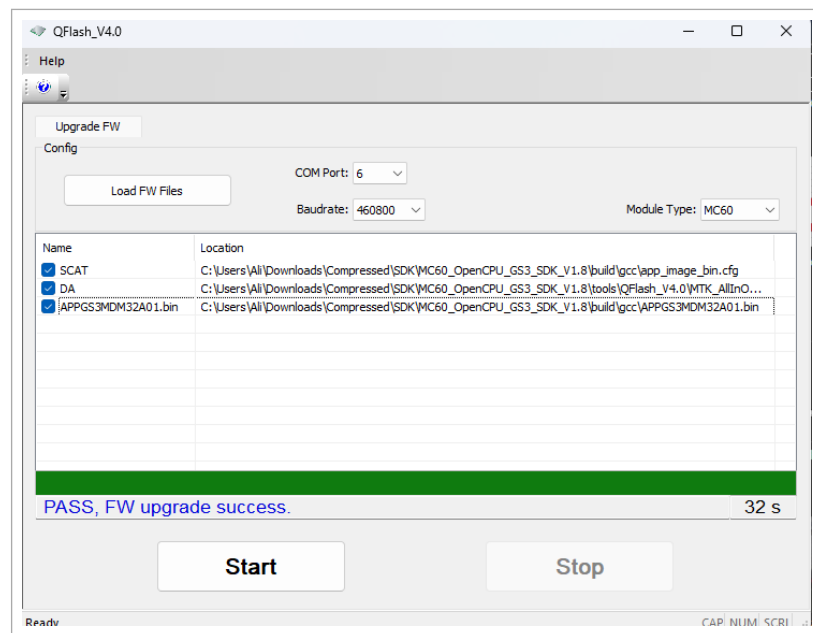


Figure 2: QFlash V4.0 reporting PASS, FW upgrade success in 32 s. The same scatter file, download agent, and module type from previous tests were reused.

5. Serial Output and Analysis

After flashing, the module was power-cycled and PuTTY (115200 baud, COM6) was opened to observe the debug output. With the SIM card inserted and GPRS connectivity available, the location demo ran to completion automatically. Figure 3 shows the full terminal capture from a single clean run.

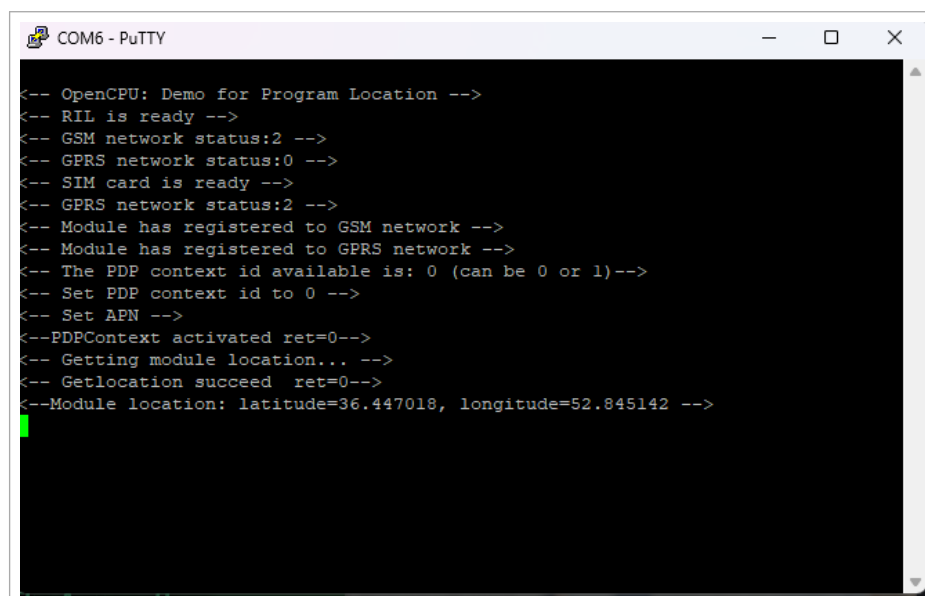


Figure 3: PuTTY terminal output from a complete single run of example_location.c, showing the full boot, network registration, PDP activation, and QuecLocator result sequence.

5.1. Boot and Network Registration

The module booted and the RIL became ready within a few seconds. GSM registration was achieved immediately (status 2 = registered, home network), confirming the SIM is provisioned for the local network. GPRS data registration followed shortly after, enabling the GPRS data bearer needed for the QuecLocator query.

The GPRS network status:0 line seen early in the boot sequence is normal: it reflects the state before GPRS attachment completes. The code polls and waits until status 2 is achieved before proceeding.

5.2. PDP Context and QuecLocator

Once GPRS was registered, the example configured and activated PDP context 0. The APN was set automatically from the SIM/network defaults. PDPContext activated ret=0 confirms the data bearer was established successfully (return value 0 = no error).

The QuecLocator query was then sent to Quectel's positioning server. Getlocation succeed ret=0 confirms the server responded with a valid position fix.

5.3. Location Result

The full serial output from a single representative run is reproduced below:

```
<-- OpenCPU: Demo for Program Location -->
<-- RIL is ready -->
<-- GSM network status:2 -->
<-- GPRS network status:0 -->
<-- SIM card is ready -->
<-- GPRS network status:2 -->
<-- Module has registered to GSM network -->
<-- Module has registered to GPRS network -->
<-- The PDP context id available is: 0 (can be 0 or 1)-->
<-- Set PDP context id to 0 -->
<-- Set APN -->
<--PDPContext activated ret=0-->
<-- Getting module location... -->
<-- Getlocation succeed ret=0-->
<--Module location: latitude=36.447018, longitude=52.845142 -->
```

Table 1 provides a line-by-line analysis of the output sequence:

Table 1: Serial output line-by-line analysis

Serial line	Meaning
<-- OpenCPU: Demo for Program Location -->	Example header — firmware is running
<-- RIL is ready -->	Radio Interface Layer initialised
<-- GSM network status:2 -->	GSM registered on home network (status 2)
<-- GPRS network status:0 -->	GPRS not yet attached
<-- SIM card is ready -->	SIM detected and PIN-free
<-- GPRS network status:2 -->	GPRS registered on home network
<-- Module has registered to GSM network -->	GSM registration confirmed
<-- Module has registered to GPRS network -->	GPRS registration confirmed

Serial line	Meaning
<-- The PDP context id available is: 0 -->	PDP bearer ID 0 selected
<-- Set PDP context id to 0 -->	PDP context configured
<-- Set APN -->	APN written to context
<-- PDPContext activated ret=0 -->	Data bearer active (ret=0 = success)
<-- Getting module location... -->	QuecLocator query sent to server
<-- Getlocation succeed ret=0 -->	Server responded successfully
<-- Module location: lat=36.447018, lon=52.845142 -->	Cell-tower position returned

The example ran successfully twice in a row (both runs visible in Figure 4 side-by-side), returning the identical coordinates each time, which confirms the result is stable and repeatable rather than a one-off artefact.

6. Location Verification

The returned coordinates were verified against Google Maps. The values are already in decimal degrees and require no conversion:

```
Latitude: 36.447018 deg N
Longitude: 52.845142 deg E
```

Entering these coordinates into Google Maps confirmed the location as **Sari, Mazandaran Province, Iran** — consistent with the GPS fix obtained in the third test (36.44382° N, 52.84479° E) and with the expected physical location of the hardware. The QuecLocator result is displaced by approximately 80 m from the GPS fix, which is well within the expected accuracy range of cell-tower triangulation.

Location Summary	
Coordinates (returned)	36.447018 N, 52.845142 E
Method	Cell-tower triangulation (QuecLocator)
Google Maps result	Sari, Mazandaran Province, Iran
GPS fix (Test 3)	36.44382 N, 52.84479 E (GNSS, HDOP 1.85)
Displacement vs GPS	approx. 80 m (within expected cell-tower accuracy)
Repeatability	Identical result on two consecutive runs

Figures 4 and 5 show the Claude.ai conversation used to verify the result and display it on a Google Maps embed.

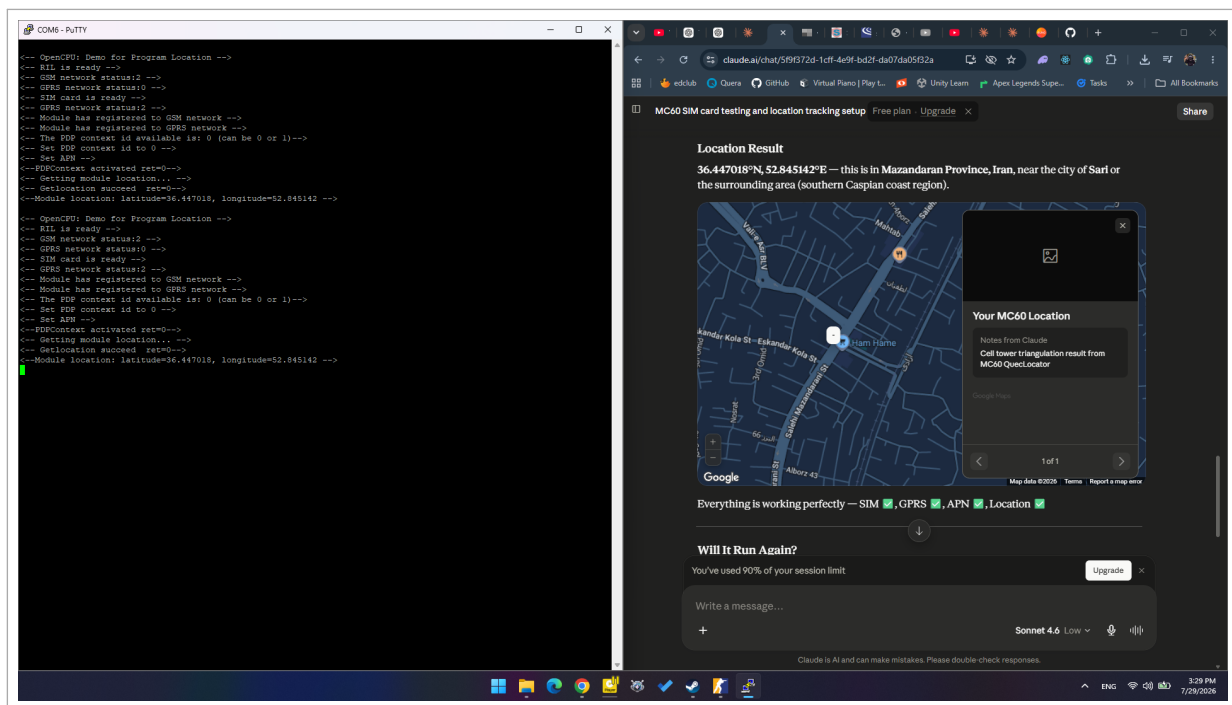


Figure 4: PuTTY terminal (left) showing two consecutive successful runs alongside the Claude.ai conversation (right) confirming the location as Sari, Mazandaran Province, Iran, with all subsystems passing: SIM ✓, GPRS ✓, APN ✓, Location ✓.

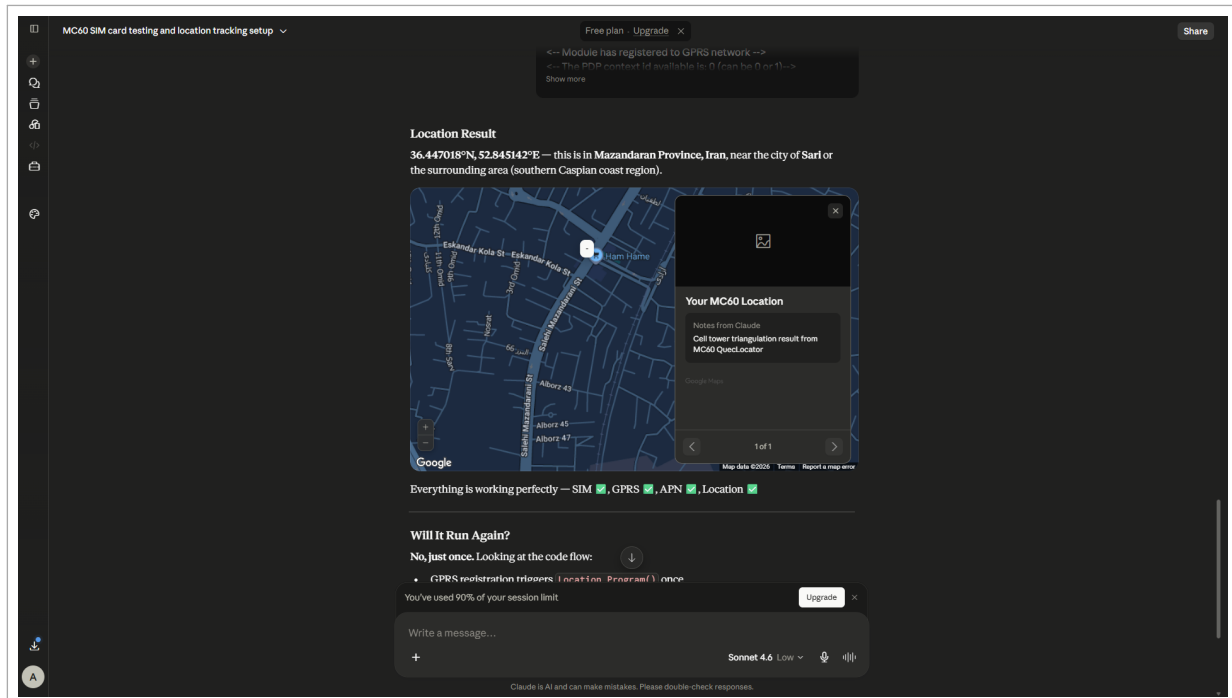


Figure 5: Google Maps embed in Claude.ai pinning the QuecLocator result (36.447018° N, 52.845142° E) in Sari, Iran. The note reads “Cell tower triangulation result from MC60 QuecLocator”.

7. Summary

Table 2 summarises the steps completed and the outcome of each.

Table 2: Test summary

Step	Notes	Status
Switch example	C_PREDEF changed to __EXAMPLE_LOCATION__	Done
Build	Make.bat new, same toolchain, GCC Compiling Finished	Done
Flash	QFlash V4.0, COM6, MC60, PASS in 32 s	Done
PuTTY monitor	COM6, 115200 baud	Done
SIM / GSM	Registered on home network (status 2)	Done
GPRS / PDP	GPRS registered, PDP context 0 activated (ret=0)	Done
QuecLocator query	Getlocation succeed ret=0	Done
Location result	36.447018° N, 52.845142° E	Done
Verification	Google Maps confirms Sari, Mazandaran Province, Iran	Done
Repeatability	Identical result on two consecutive runs	Done

Result

The fourth test is considered fully successful. The example_location.c demo correctly registered on the GSM and GPRS networks, activated the GPRS data bearer, contacted the QuecLocator server, and returned a valid cell-tower position. The coordinates (36.447018 N, 52.845142 E) match the expected physical location in Sari, Iran, and agree with the GPS fix from the third test to within 80 m. The result was reproducible across two consecutive runs without any firmware or configuration changes.

References

- [1] Quectel, *MC60 OpenCPU GS3 SDK V1.8*, SDK release package — example_location.c.
- [2] Quectel, *QuecLocator Application Note*, Quectel documentation.
- [3] Arm Ltd., *Arm GNU Toolchain 15.2.1 (mingw-w64-i686-arm-none-eabi)*, <https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>.
- [4] First Test Report — MC60 OpenCPU GS3 SDK build and GPS example (no fix), June 2026.
- [5] Second Test Report — MC60 OpenCPU GS3 SDK Bluetooth example, June 2026.
- [6] Third Test Report — MC60 OpenCPU GS3 SDK GPS example (valid fix), July 2026.
- [7] *MC60_Build_Changes.pdf* — companion makefile fix documentation, June 2026.